Lógica de Programação | Aula 06

Iteração I: Laços Determinados (for e range)

João Quentino

05 de Março de 2026

Jovem Tech | Turma 04A - Manhã

Recapitulação da Aula 05

O que aprendemos na última aula:

Estruturas Condicionais:

- IF - decisão simples
- IF-ELSE - duas opções
- IF-ELIF-ELSE - múltiplas opções

Conceitos Importantes:

- Indentação obrigatória
- Aninhamento de condições
- Boas práticas de código

Hoje: Vamos aprender a fazer o programa REPETIR ações!

Agenda de Hoje

Introdução aos Loops

O Loop FOR

A Função range()

Padrões Comuns de Loops

Iterando sobre Sequências

Loops Aninhados

Exercícios Práticos

Boas Práticas com Loops

Recapitulação

Introdução aos Loops

O que são Loops?

Loops (laços) permitem repetir ações automaticamente.

Na vida real:

- Escovar cada dente (repetir para cada dente)
- Lavar cada prato (repetir para cada prato)
- Contar de 1 a 100 (repetir a contagem)

Na programação:

- Exibir mensagem 10 vezes
- Somar números de 1 a 100
- Processar cada item de uma lista
- Desenhar padrões repetitivos

Sem loops: Código gigante e repetitivo
Com loops: Código curto e elegante!

Loops no Scratch vs Python

Loops do Scratch!

Scratch:

- Repita 10 vezes
- Sempre (forever)
- Repita até que

Blocos visuais de repetição

Python:

- `for` (repita N vezes)
- `while` (repita até que)

Comandos textuais

Hoje: Foco no loop `for` (laços determinados)
Próxima aula: Loop `while` (laços indeterminados)

Laços Determinados vs Indeterminados

Duas categorias de loops:

Determinados (FOR):

- Sabemos QUANTAS vezes repetir
- Número fixo de iterações
- Exemplo: Repetir 10 vezes
- Exemplo: Para cada item da lista

Aula de hoje!

Indeterminados (WHILE):

- NÃO sabemos quantas vezes
- Repete até condição mudar
- Exemplo: Até acertar senha
- Exemplo: Enquanto há saldo

Próxima aula!

O Loop FOR

Estrutura Básica do FOR

Sintaxe:

```
for variavel in sequencia:
    # Codigo repetido para cada item
    acao1
    acao2
    ...
```

Elementos:

- `for` - palavra-chave
- `variavel` - recebe cada item da sequência
- `in` - palavra-chave
- `sequencia` - o que será iterado
- `:` - dois pontos obrigatórios
- **Indentação** - bloco do loop

Primeiro Exemplo: Loop Simples

Exibir mensagem 5 vezes:

```python
for i in range(5):
    print("Ola, Python!")

print("Loop finalizado!")
```

Saída:

```
Ola, Python!
Ola, Python!
Ola, Python!
Ola, Python!
Ola, Python!
Loop finalizado!
```

Usando a Variável do Loop

A variável `i` guarda o valor atual:

```python
for i in range(5):
    print(f"Iteracao numero: {i}")
```

Saída:

```
Iteracao numero: 0
Iteracao numero: 1
Iteracao numero: 2
Iteracao numero: 3
Iteracao numero: 4
```

Observe: Python começa do 0! (índices começam em 0)

A Função range()

O que é range()?

range() gera uma sequência de números.

Características:

- Função built-in do Python
- Gera números inteiros
- Muito eficiente (não cria lista na memória)
- Essencial para loops FOR

Três formas de usar:

1. `range(stop)` - do 0 até stop-1
2. `range(start, stop)` - de start até stop-1
3. `range(start, stop, step)` - com passo customizado

range() com 1 Parâmetro

range(stop) - começa em 0, vai até stop-1:

```python
# range(5) gera: 0, 1, 2, 3, 4
for i in range(5):
    print(i)
```

Saída: 0, 1, 2, 3, 4

```python
# range(3) gera: 0, 1, 2
for numero in range(3):
    print(f"Numero: {numero}")
```

Importante: `range(5)` gera 5 números (0 a 4), não até 5!

range() com 2 Parâmetros

range(start, stop) - começa em start, vai até stop-1:

```python
# range(1, 6) gera: 1, 2, 3, 4, 5
for i in range(1, 6):
    print(i)
```

Saída: 1, 2, 3, 4, 5

```python
# range(10, 15) gera: 10, 11, 12, 13, 14
for numero in range(10, 15):
    print(f"Contando: {numero}")
```

Útil para: Contar de 1 a 10, de 5 a 20, etc.

range() com 3 Parâmetros

range(start, stop, step) - define o passo:

```python
# Numeros pares de 0 a 10
for i in range(0, 11, 2):
    print(i)  # 0, 2, 4, 6, 8, 10
```

```python
# Contar de 5 em 5
for i in range(0, 51, 5):
    print(i)  # 0, 5, 10, 15, 20, ..., 50
```

```python
# Contagem regressiva!
for i in range(10, 0, -1):
    print(i)  # 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
```

Visualizando range()

Resumo visual dos parâmetros:

Código	Números Gerados
`range(5)`	0, 1, 2, 3, 4
`range(1, 5)`	1, 2, 3, 4
`range(2, 10, 2)`	2, 4, 6, 8
`range(10, 0, -2)`	10, 8, 6, 4, 2
`range(5, 0, -1)`	5, 4, 3, 2, 1

Dica: Para contar de 1 a N, use `range(1, N+1)`

Padrões Comuns de Loops

Padrão 1: Acumulador (Soma)

Somar números de 1 a 100:

```python
soma = 0  # Inicializa acumulador

for i in range(1, 101):
    soma = soma + i  # Ou: soma += i

print(f"Soma de 1 a 100: {soma}")  # 5050
```

Padrão Acumulador:

1. Inicializar variável antes do loop
2. Dentro do loop: acumular valores
3. Após o loop: usar o resultado

Padrão 2: Contador

Contar quantos números pares de 1 a 50:

```python
contador = 0

for numero in range(1, 51):
    if numero % 2 == 0:
        contador += 1

print(f"Total de pares: {contador}")  # 25
```

Diferença:

- **Acumulador:** soma/multiplica valores
- **Contador:** apenas conta (+1)

Padrão 3: Busca

Encontrar o primeiro múltiplo de 7 maior que 50:

```python
encontrado = False

for numero in range(51, 100):
    if numero % 7 == 0:
        print(f"Encontrado: {numero}")
        encontrado = True
        break  # Para o loop!

if not encontrado:
    print("Nao encontrado")
```

break: Interrompe o loop imediatamente
Útil para: Quando encontra o que procura

Padrão 4: Validação

Verificar se todos os números são positivos:

```python
numeros = [5, 10, -3, 8, 15]
todos_positivos = True

for numero in numeros:
    if numero < 0:
        todos_positivos = False
        break

if todos_positivos:
    print("Todos sao positivos!")
else:
    print("Tem numero negativo!")
```

Continue: Pular Iteração

continue pula para a próxima iteração:

```
# Imprimir apenas numeros impares
for i in range(10):
    if i % 2 == 0:
        continue  # Pula pares
    print(i)  # Imprime apenas impares
```

Saída: 1, 3, 5, 7, 9

Diferença:

- **break:** para o loop completamente
- **continue:** pula para próxima iteração

Iterando sobre Sequências

Iterando sobre Strings

Percorrer cada caractere de uma string:

```python
palavra = "Python"

for letra in palavra:
    print(letra)
```

Saída:

```
P
y
t
h
o
n
```

Exemplo: Contar Vogais

Contar quantas vogais tem uma palavra:

```python
texto = input("Digite uma palavra: ")
vogais = "aeiouAEIOU"
contador = 0

for letra in texto:
    if letra in vogais:
        contador += 1

print(f"Total de vogais: {contador}")
```

Conceito novo: `in` para verificar se está contido!

Loops Aninhados

O que são Loops Aninhados?

Loop dentro de outro loop:

```python
for i in range(3):
    for j in range(4):
        print(f"i={i}, j={j}")
```

Saída: 12 linhas (3 x 4)

```
i=0, j=0
i=0, j=1
i=0, j=2
i=0, j=3
i=1, j=0
...
```

Exemplo: Tabuada

Gerar tabuada de 1 a 10:

```python
for numero in range(1, 11):
    print(f"\nTabuada do {numero}:")
    for multiplicador in range(1, 11):
        resultado = numero * multiplicador
        print(f"{numero} x {multiplicador} = {resultado}")
```

Estrutura:

- Loop externo: escolhe o número (1 a 10)
- Loop interno: multiplica por 1 a 10

Exemplo: Padrão de Estrelas

Desenhar triângulo de asteriscos:

```python
linhas = 5

for i in range(1, linhas + 1):
    for j in range(i):
        print("*", end="")
    print()  # Nova linha
```

Saída:

```
*
**
***
****
*****
```

Exemplo: Matriz (Grade)

Criar uma grade numérica:

```
for linha in range(1, 4):
    for coluna in range(1, 6):
        print(f"[{linha},{coluna}]", end=" ")
    print()  # Nova linha
```

Saída:

```
[1,1] [1,2] [1,3] [1,4] [1,5]
[2,1] [2,2] [2,3] [2,4] [2,5]
[3,1] [3,2] [3,3] [3,4] [3,5]
```

Exercícios Práticos

Exercício 1: Contagem Regressiva

Desafio (5 min):

Faça um programa que imprima uma contagem regressiva de 10 até 0.

Dica:

- Use um `for` com `range()`.
- Lembre que o `range(inicio, fim, passo)` permite definir um passo negativo.
- Para contar para trás, use passo **-1**.
- O número final do `range` não é incluído.

Após a contagem, imprima a mensagem:

```
1 print("Feliz Ano Novo!")
```

Saída esperada: 10, 9, 8, ..., 1, 0, Feliz Ano Novo!

Exercício 2: Fatorial

Desafio (10 min):

O fatorial de um número é definido como:

$$n! = n \times (n-1) \times (n-2) \times ... \times 1$$

Exemplo:

- $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Escreva um programa que:

- Leia um número inteiro
- Multiplique todos os números de 1 até esse número
- Mostre o resultado

Teste: $5! = 120$, $6! = 720$

Exercício 3: Múltiplos

Desafio (10 min):

Mostre todos os múltiplos de 3 entre 1 e 50.

Um número é múltiplo de 3 quando o resto da divisão por 3 é igual a zero.

$$numero\%3 == 0$$

Passos:

- Percorra os números de 1 até 50 usando `range()`
- Para cada número, verifique se ele é divisível por 3
- Se for, imprima o número

Saída esperada:

3, 6, 9, 12, 15, 18, ..., 48

Exercício 4: Números Primos

Desafio (15 min): Um número primo é um número que só pode ser dividido por:

- 1
- ele mesmo

Exemplos de números primos: 2, 3, 5, 7, 11, 13

Ideia do algoritmo:

- Leia um número
- Teste se ele é divisível por algum número entre 2 e $n - 1$
- Se encontrar um divisor, ele **não é primo**
- Caso contrário, ele é primo

Dica:

Use um `for` para testar divisões e uma variável booleana para guardar o resultado.

Exercício 5: Sequência de Fibonacci

Desafio (15 min):

A sequência de Fibonacci é formada somando sempre os dois números anteriores. Exemplo: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34 ...

Regra da sequência:

$$proximo = anterior + atual$$

Como fazer:

- Comece com dois valores iniciais: 0 e 1
- Calcule o próximo número somando os dois
- Atualize os valores
- Repita o processo usando um `for`

O programa deve gerar os primeiros **N números da sequência**.

Exercício 6: Pirâmide de Números

Desafio (15 min):

Desenhe uma pirâmide de números baseada em uma altura informada pelo usuário.

Exemplo para altura = 4:

```
   1
  12
 123
1234
```

Dica:

Use dois loops aninhados:

- Um para imprimir os espaços
- Outro para imprimir os números

Exercício 7: Validador de Senha

Desafio (15 min):

Crie um programa que valide uma senha com as seguintes regras:

- Deve ter pelo menos 8 caracteres
- Deve ter pelo menos uma letra maiúscula
- Deve ter pelo menos uma letra minúscula
- Deve ter pelo menos um número

Como resolver:

- Percorra cada caractere da senha usando um `for`
- Verifique se ele é:
  - maiúsculo
  - minúsculo
  - número

Use variáveis booleanas para guardar essas verificações e depois combine todas as condições para decidir se a senha é válida.

Boas Práticas com Loops

Boas Práticas

Dicas para loops eficientes e legíveis:

1. **Nomes de variáveis descritivos**
   - `i, j, k` para contadores simples
   - `aluno, numero, letra` para itens específicos
2. **Evite modificar a variável do loop**
   - Pode causar bugs difíceis de encontrar
3. **Use break com moderação**
   - Deixe claro quando e por que sai do loop
4. **Limite aninhamento**
   - Máximo 2-3 níveis
   - Considere criar funções

Comparando Estilos

Ruim:

```
for x in range(10):
    if x%2==0:
        for y in range(5):
            if y>2:
                print(x*y)
```

Confuso, difícil de ler

Bom:

```
for numero in range(10):
    if numero % 2 == 0:
        for mult in range(3, 5):
            resultado = numero * mult
            print(resultado)
```

Claro e descritivo

Recapitulação

O que Aprendemos Hoje

Loops FOR e range():

1. **Loop FOR**: Repetição determinada
2. **range()**: Gera sequências numéricas
3. **Padrões**: Acumulador, contador, busca
4. **break/continue**: Controle do loop
5. **Iteração**: Strings, listas, sequências
6. **Loops aninhados**: FOR dentro de FOR

Agora você domina repetições determinadas!

Próximo: Loop WHILE (repetições indeterminadas)

Desafio para Casa

Projetos para praticar:

1. **Gerador de Tabuada Completa:**
   - Todas as tabuadas de 1 a 10
   - Formatado e organizado
2. **Conversor de Binário:**
   - Converta número decimal para binário
   - Use divisões sucessivas por 2
3. **Jogo de Adivinhação:**
   - Computador sorteia número
   - Usuário tem 5 tentativas
   - Use FOR com range(5)

Dúvidas e Encerramento

Momento para perguntas!

Contato:

joao.quentino@dcomp.ufs.br

Obrigado!

Na próxima aula: Loop WHILE (Iteração II)!